

Fast Pattern Searching in a String

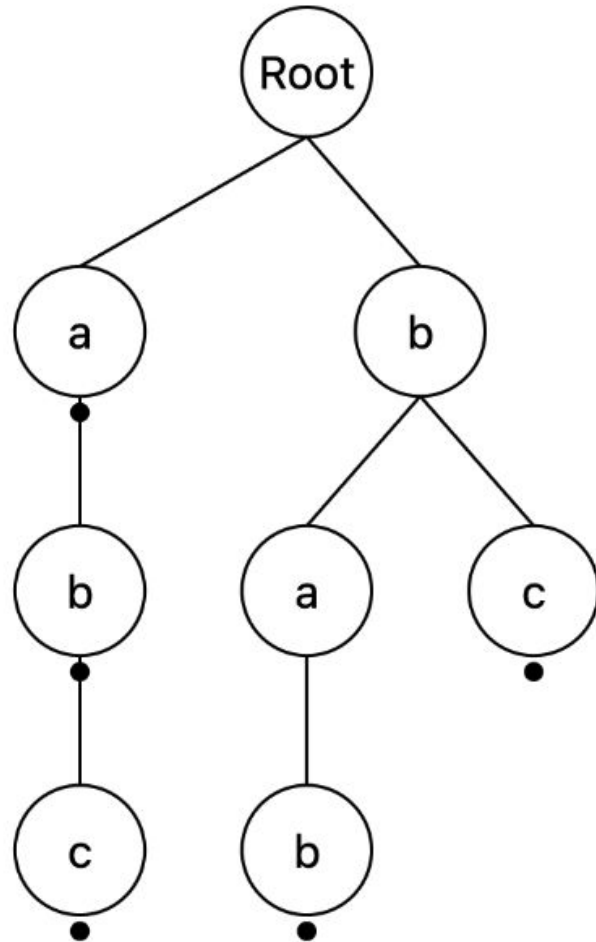
By Nandan and Suraj

Introduction/Problem Statement

- Given a string of specified length and a set of patterns , it is often required to find the number of occurrences of the patterns in the given text string .
- For short strings, a *brute-force approach* (often called the *naive shift-based pattern matching algorithm*) can be used to count pattern occurrences .
- However, in real-life applications such as text searching — the text length is often very large, making brute-force methods inefficient .
- This motivates the use of the *Aho–Corasick algorithm*, which provides a faster and more efficient way to find multiple patterns simultaneously .

Aho-Corasick algorithm and how it works

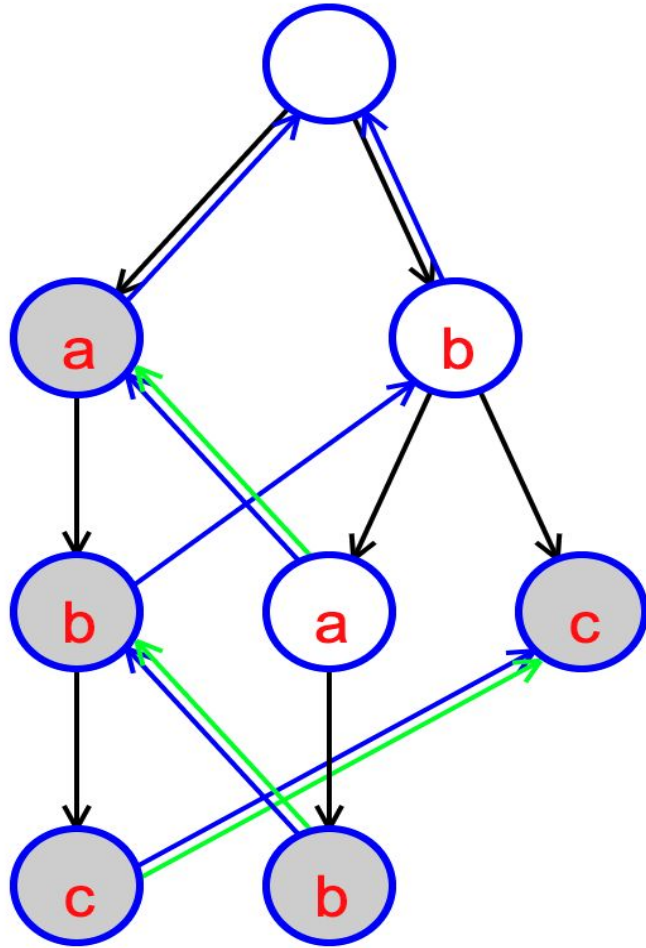
- The **Aho–Corasick algorithm** is a **string-searching algorithm** used to find multiple patterns within a text simultaneously .
- It is based on a data structure called **Keyword Tree (Prefix Tree / Trie)**.
- The algorithm builds a **Prefix Tree / Trie** from all given patterns to represent them efficiently .
- For example:
Consider a text string **“ababcab”** and a set of patterns {**“a”**, **“ab”**, **“bab”**, **“bc”**, **“abc”**} .



This figure shows **Trie**
construction for the text string
“ababcb”

- It then adds failure links to handle mismatches quickly during the search and adds output links to connect the nodes that correspond to overlapping or embedded patterns .
- Once built, the algorithm scans the text only once to find all pattern occurrences .

- Black arrows represent the normal Trie connections between characters .
- Blue arrows show the failure links, which redirect the search when a mismatch occurs .
- Green arrows indicate the output links, connecting nodes that correspond to overlapping or embedded patterns .



This figure shows **Automaton** construction for the text string **“ababcbab”**

Pseudo code

Node structure:

- next[] → transitions
- fail → failure link
- out → pattern IDs ending here

Trie initialization:

- Start with root node

insertPattern(pattern, id):

- Add pattern to trie
- Mark node with pattern ID

buildAutomaton():

- Set root's children fail links
- BFS to set fail links for all nodes
- Propagate output lists along fail links

searchText(text):

- Follow transitions for each character
- Count occurrences of patterns from output lists

Main:

- Insert all patterns
- Build automaton
- Search text and print pattern counts

Statistics

- Given a text string of length N and a set of patterns P , let L denote the total cumulative length of all the patterns combined , and K denote the total number of pattern occurrences found in the text .
- The total time complexity of the algorithm is $O(N+L+K)$.

$O(N)$	$O(N)$ is the time spent travelling the text, performing one state transition for each of the N characters
$O(L)$	$O(L)$ is the dominant time spent building the initial Tree and the failure function
$O(K)$	$O(K)$ is the time spent reporting the results. This counts every single match, including overlapping matches and matches triggered via the failure links.

Comparison with naive-shift based algorithm

How the naive-shift based algorithm works

- Given a text string T and a set of Patterns P , we first take the first pattern and align it with the beginning of the text .
- We then compare each character of the pattern with the corresponding character in the text .
- If all the characters match, we record that position as a valid occurrence .
- If a mismatch occurs, we shift the pattern one position to the right and repeat the comparison .
- This process continue until the end of the text is reached .
- The same process is then repeated for each pattern in P .

- The run time for the naive-shift based algorithm is $O(N.L)$, where N is the length of the text string and L is the combined length of all the patterns .

Let's compare the run time between naive-shift based algorithm and Aho-Corasick algorithm for some examples

- ❖ Text string = “**ababcab**”, Patterns P = {“**a**”, “**ab**”, “**bab**”, “**bc**”, “**abc**”}

	Aho-Corasick Algorithm	Naive shift based algorithm
Run time (micro sec)	45-105	20-50

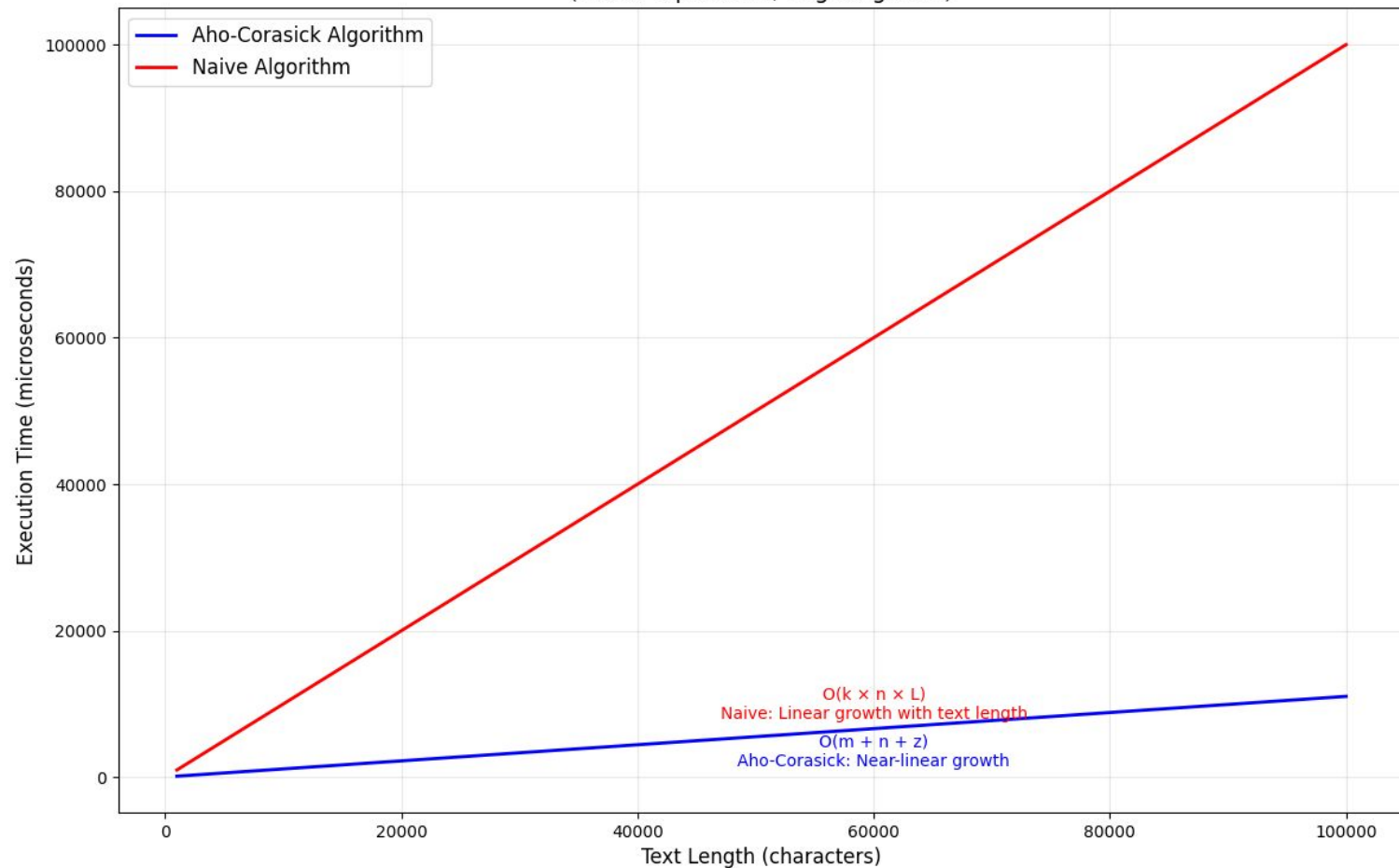
- ❖ Text string = **the string with 2261 characters**
- ❖ Pattern P={“AGCT”, “GCTA”, “TACG”, “CGTAG”, “ACCTAG”}

	Aho-Corasick Algorithm	Naive shift based algorithm
Run time (micro sec)	240-280	1900-2400

- ❖ Text string = **complete human genome sequence (3.2 billion characters)**
- ❖ For $M = 1000$, $L = 2 \times 10^7$

	Aho-Corasick Algorithm	Naive shift based algorithm
Run time	~6 mins	~40 days

Pattern Matching: Aho-Corasick vs Naive Algorithm
(Fixed: 5 patterns, avg length=4)



Comparison with suffix tree

- Aho-Corasick algorithm uses patterns to build the tree, but the suffix tree uses text string to build the tree .
- The inputs for the Prefix Tree are patterns and text string, but for the suffix tree the inputs is only text string.

	Prefix Tree	Suffix Tree
Time complexity	$O(N+L+K)$	$O(N+K)$

	Prefix Tree	Suffix Tree
Space complexity	$O(L)$	$O(N)$

Real life based applications

❖ Bioinformatics

- Used for DNA and protein sequence analysis to find multiple gene patterns simultaneously
- Example: Searching thousands of known DNA sequences in a large genome database

❖ Natural Language Processing (NLP)

- Used for named entity recognition (NER) –identifying names, locations, or organizations in text
- Common in chatbots and AI text analyzers to quickly locate predefined keywords or phrases

❖ Search Engines and Text Processing

- Used in auto-suggestion and keyword highlighting when typing search queries.
- Helps find all occurrences of multiple search terms efficiently in large documents.

❖ Network Security / Intrusion Detection

- Implemented in systems like **Snort** and **Suricata** for real-time packet inspection.
- Helps detect if any known attack patterns (signatures) appear in network traffic.

❖ Plagiarism and Content Matching

- Compares a document against multiple source simultaneously.
- Used plagiarism detection tools and content moderation systems.

❖ Spam and Malware Detection

- Email filters use it to scan incoming emails for multiple spam keywords or phrases.
- Antivirus software uses it to detect malware signatures in files and network packets.

References

- Aho, A. V., & Corasick, M. J. (1975). *Efficient String Matching: An Aid to Bibliographic Search*. *Communications of the ACM*, 18(6), 333–340.
<https://cr.yp.to/bib/1975/aho.pdf>
- Subramanian, A. (2021, April 13). *Aho–Corasick Algorithm | Pattern Searching Algorithm*. Medium.
<https://medium.com/pattern-searching-algorithm/aho-corasick-algorithm-7e5c2e58861c>